An

Internship Project Report

On

Email Spam and Scam Detection System

Using Machine Learning and Natural Language Processing

Submitted in partial fulfilment of the requirements

for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by

Shani Chauhan (25mc201A61)

Under the guidance of

[GUIDE NAME]

[Designation]

DEPARTMENT OF INFORMATION TECHNOLOGY

ANURAG UNIVERSITY

(Approved by AICTE and NBA Accredited)

Venkatapur (V), Ghatkesar (M), Medchal District, Hyderabad, Telangana, 500088

Academic Year 2026 2027

Batch 2025 2027

29 October 2026

CERTIFICATE

ANURAG UNIVERSITY

DEPARTMENT OF INFORMATION TECHNOLOGY

This is to certify that the internship project report entitled “Email Spam and Scam Detection System Using Machine Learning and Natural Language Processing” is a bonafide work carried out and submitted by Shani Chauhan (25mc201A61) in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications at Anurag University during the batch 2025–2027.

The work was carried out under the guidance of [GUIDE NAME].

Internal Guide	**Head of the Department**
[GUIDE NAME]	[HOD NAME]
[Designation]	[Designation]

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, [GUIDE NAME], Department of Information Technology, Anurag University, for guidance, suggestions and encouragement during this project.

I thank [HOD NAME], Head of the Department, and the MCA programme coordinator for their academic support. I also thank the faculty members who supported my learning and development throughout the programme.

Finally, I thank my family and friends for their encouragement and support during the preparation of this project and report.

Shani Chauhan (25mc201A61)

DECLARATION

I, Shani Chauhan (25mc201A61), declare that the project report entitled “Email Spam and Scam Detection System Using Machine Learning and Natural Language Processing” is my own work, submitted in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications at Anurag University, Hyderabad.

I declare that sources used in this report have been acknowledged in the references and that this work has not been submitted previously for the award of another degree.

Place Hyderabad

Date _____

Signature _____

Shani Chauhan (25mc201A61)

ABSTRACT

SpamGuard AI is a Streamlit application for classifying message text and explaining warning indicators. The trained machine learning pipeline uses TF IDF features with Multinomial Naive Bayes and a calibrated linear support vector machine. Separate rules examine suspicious links, sensitive information requests, recruitment payments and changes in conversation details.

The revised experiment uses CEAS 08 email data from the Kaggle Phishing Email Dataset and the UCI SMS Spam Collection. From 44,728 input records, cleaning and normalized text deduplication produced 37,847 usable examples. The training partition contains 30,277 messages and the separate test partition contains 7,570. SVM achieved 98.35 percent test accuracy, 98.00 percent spam precision and 98.15 percent spam recall; Naive Bayes achieved 94.87 percent accuracy.

The report documents dataset provenance, preprocessing, training, evaluation and application testing. All 143 automated application tests passed with the new models active. Results describe historical English spam classification and do not establish performance on present day scams, Hindi or Hinglish. Near duplicate messages and campaign overlap may remain after exact normalized text deduplication.

TABLE OF CONTENTS

1 Introduction.....9
2 Background and objectives.....10
3 Dataset and methodology.....11
4 Implementation.....13
5 Performance evaluation.....14
6 Application testing.....16
7 Conclusion and future scope.....18
References.....19
Appendix A Reproducibility record.....20

LIST OF TABLES AND FIGURES

Table 1 Input dataset counts.....11

Table 2 Data preparation audit.....11

Table 3 Training configuration.....12

Table 4 Application modules.....13

Table 5 Held out model performance.....14

Table 6 Prediction error counts.....15

Table 7 Test coverage and boundaries.....17

Figures

Figure 1 Model performance comparison.....14

Figure 2 Model confusion matrices.....15

Figure 3 Automated application test results.....16

LIST OF ABBREVIATIONS

AI Artificial Intelligence

API Application Programming Interface

CSV Comma Separated Values

F1 Harmonic mean of precision and recall

FPR False Positive Rate

HTML Hypertext Markup Language

ML Machine Learning

MNB Multinomial Naive Bayes

NLP Natural Language Processing

OCR Optical Character Recognition

ROC AUC Area Under the Receiver Operating Characteristic Curve

SVM Support Vector Machine

TF IDF Term Frequency Inverse Document Frequency

UI User Interface

1 Introduction

1.1 Project overview

Email and short messages carry legitimate communication alongside unsolicited advertising, phishing attempts and social engineering. SpamGuard AI helps a user inspect a message, understand warning indicators and decide which verification action to take. Its three main application steps are Check, Understand and Action.

This revision reports the training completed on 29 October 2026 using the two supplied dataset archives. It replaces the earlier unverified performance figures with recorded test results from a reproducible two class experiment. SVM performed best among the two trained models, with 98.35 percent accuracy on 7,570 held out messages.

1.2 Problem statement

A keyword warning alone may miss a harmful message or flag a genuine urgent request. A text classifier can recognize statistical patterns, while explicit rules can explain specific concerns. The system combines these forms of evidence but does not verify that a sender is authentic or determine whether a message constitutes fraud.

1.3 Scope of the report

The experiment measures binary ham versus spam classification on historical English email and SMS data. Ham means a legitimate message label; spam means the source dataset marks the message as spam. These labels are not a complete taxonomy of confirmed financial scams. Dataset references appear in the methodology and reference list [1, 2].

1.4 Main outcomes

The supplied archives contained 44,728 records. Removing unusable content and duplicate normalized text left 37,847 examples. Training used 30,277 examples, with 7,570 reserved for testing. Both Naive Bayes and SVM were saved in a versioned model release. The project test suite passed all 143 tests after activation, and the previous model release was preserved for rollback.

2 Background and objectives

2.1 Text classification methods

TF IDF represents messages as weighted terms. Terms that occur frequently within a message but less frequently across the corpus receive more influence than common terms. The project uses word unigrams and bigrams so that individual words and short phrases can contribute to a prediction [3].

Multinomial Naive Bayes provides a compact baseline suited to nonnegative count or TF IDF features. Linear SVM learns a separating decision boundary in the sparse text feature space. The project wraps LinearSVC in sigmoid probability calibration with three internal folds [4, 5].

2.2 Classification and explanation

The trained classifier predicts spam or ham from cleaned text. A separate assessment layer considers links, suspicious requests and contextual evidence. Consequently, a classifier probability and the application risk score represent different outputs. A high benchmark score for the classifier does not validate every rule or every final risk assessment.

2.3 Project objectives

- Train both existing classifier families using the supplied CEAS and SMS datasets.
- Prevent identical normalized messages from entering both training and test partitions.
- Report accuracy, spam precision, recall, F1, ROC AUC and confusion matrices from a fixed test set.
- Check that model activation preserves the existing application workflows and supports rollback.
- Document dataset provenance and limitations so another student can reproduce the experiment.

2.4 Evaluation approach

The training design uses a fixed 80 to 20 split stratified by source and label, with random seed 42. TF IDF is fitted only on the training partition. The experiment does not report five fold outer cross validation, ensemble results, Random Forest results or an independent modern scam benchmark.

3 Dataset and methodology

3.1 Dataset sources

CEAS_08.csv was supplied in CEAS_08.csv.zip and obtained from the Kaggle Phishing Email Dataset collection [1]. Only this component of that collection was used. The archive contains 39,154 email records with sender, receiver, date, subject, body, label and urls fields. Subject and body are joined as model input; sender, receiver, date and the URL count field are excluded.

The second archive, sms+spam+collection.zip, contains the UCI SMSSpamCollection file and its readme [2]. The parsed file contains 5,574 messages with a ham or spam label followed by a tab and the message text. The email and SMS sources are combined for this experiment, while retaining a source column for split stratification.

Table 1 Input dataset counts

Source	Ham	Spam	Total
CEAS 08 email	17,312	21,842	39,154
UCI SMS	4,827	747	5,574
Combined input	22,139	22,589	44,728

3.2 Data quality controls

The preparation script validates label values, cleans text with the project preprocessor and removes empty normalized text. It checks whether identical normalized text carries conflicting labels, removes such records if present, and then retains only one copy of each normalized message across both sources.

Table 2 Data preparation audit

Audit item	Recorded count
Raw records	44,728
Empty or invalid records removed	9
Conflicting label records removed	0
Duplicate normalized records removed	6,872
Usable records	37,847
Usable ham and spam	21,587 ham and 16,260 spam

The separate 140 message curated ham file described in the earlier report was not included in this training run. The counts above come from the supplied archives and saved data audit, rather than from the full Kaggle collection.

3 Dataset and methodology continued

3.3 Preprocessing and partitioning

Text is lowercased; URLs, email addresses, currency symbols and standalone numbers are normalized. Punctuation is removed, and common English stopwords and short tokens are filtered. The resulting cleaned text is the input to TF IDF. This experiment uses the existing preprocessor without introducing stemming or a transformer model.

After normalized text deduplication, the 37,847 messages are split into 30,277 training examples and 7,570 test examples. The split is stratified by dataset source and label using random seed 42. An explicit assertion checks that no cleaned text occurs in both partitions. The test set contains 4,318 ham and 3,252 spam examples.

Table 3 Training configuration

Setting	Value
TF IDF features	Maximum 6,000 word features
Ngrams	Unigrams and bigrams
Term frequency	Sublinear TF enabled
Corpus stopwords	Existing project list of Enron specific markers
Naive Bayes	MultinomialNB with alpha 0.1
SVM	LinearSVC with C 1.0 and balanced class weights
SVM reproducibility	Random seed 42 and max iterations 3,000
Probability calibration	Sigmoid calibration with three internal folds
Evaluation	Separate fixed 20 percent test partition

3.4 Fitting and reproducibility

TF IDF learns its vocabulary and weights only from training text. Both classifiers are fitted using those features; the held out test text is transformed using the fitted vectorizer. Exact partitions, archive SHA 256 hashes, model files, metrics and a release manifest are saved with version ceas-sms-20261029.

No hyperparameter search or test guided threshold tuning was performed in this run. Exact normalized duplicates were removed, but near duplicate messages and messages from the same spam campaign may still cross the split. Scores therefore require confirmation on data separated by campaign, source or time before broader claims can be made.

4 Implementation

4.1 Application structure

The application uses Streamlit for the interface and Python modules for preprocessing, prediction, evidence generation and account related operations. The active model manifest selects the version used by the normal prediction loader. Training does not require private inbox data.

Table 4 Application modules

Module	Responsibility
app.py and app_pages	Check, Understand, Action, Advanced and Settings pages
src/preprocessing.py	Shared text normalization for training and prediction
src/train.py	TF IDF fitting, classifier training and metric generation
src/predict.py	Version aware model loading and message prediction
src/scam_assistant.py	Final assessment with evidence and contextual checks
src/security_analyzer.py	URL, domain and message warning indicators
src/conversation_comparison.py	Comparison of earlier and current message details
src/image_checks.py	Optional local OCR and QR extraction
src/attachment_analyzer.py	Attachment filename and metadata checks
src/active_learning.py	Reviewed feedback training and model rollback
scripts/train_uploaded_datasets.py	Explicit import, deduplication and training of supplied ZIPs

4.2 Activation and rollback

After training, the candidate models were selected through the active model manifest and the full project test suite was run. All tests passed, so the version was retained as active. Previous baseline model files remain intact, and models/previous.json records the previous release for the existing administrator rollback feature.

4.3 User workflow

A user enters text or supplies supported message content in Check, examines evidence in Understand and chooses a supported response in Action. Advanced offers the recorded model comparison and technical evaluation tools. Optional account and image features depend on their configured services and local dependencies; model test scores do not establish their live operational status.

5 Performance evaluation

5.1 Results on held out messages

Both classifiers were evaluated on the same 7,570 test examples. Spam is the positive class for precision, recall and F1. Accuracy measures correct predictions across both labels. ROC AUC uses the classifier spam probabilities. Metrics below are recorded in the release metrics file; the plotted values show percentages.

Table 5 Held out model performance

Metric	Naive Bayes	SVM
Accuracy	94.87 percent	98.35 percent
Spam precision	95.37 percent	98.00 percent
Spam recall	92.56 percent	98.15 percent
Spam F1	93.95 percent	98.08 percent
ROC AUC	0.9897	0.9981

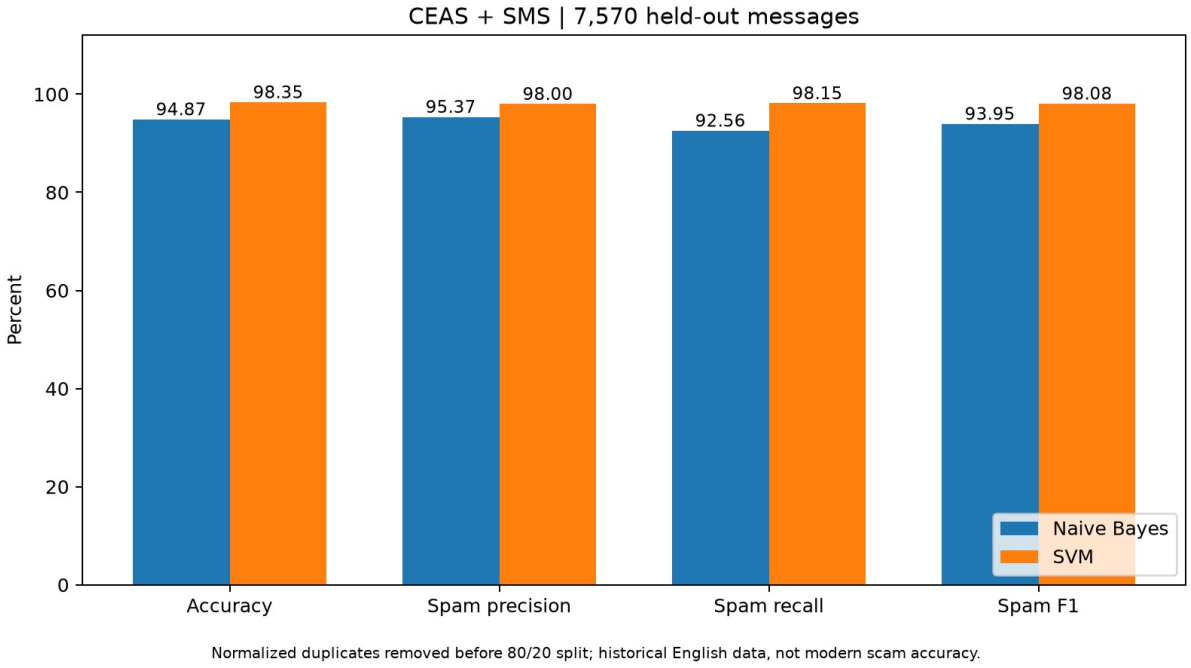


Figure 1 Model performance comparison on 7570 held out messages

SVM has higher accuracy, spam precision, recall and F1 on this split. The accuracy difference is 3.48 percentage points. This comparison concerns the statistical classifiers; the final application assessment can also incorporate rule based evidence.

5 Performance evaluation continued

5.2 Confusion matrices and error analysis

A true negative is a ham message correctly classified as ham. A false positive is a ham message classified as spam. A false negative is a spam message classified as ham. A true positive is a spam message correctly classified as spam.

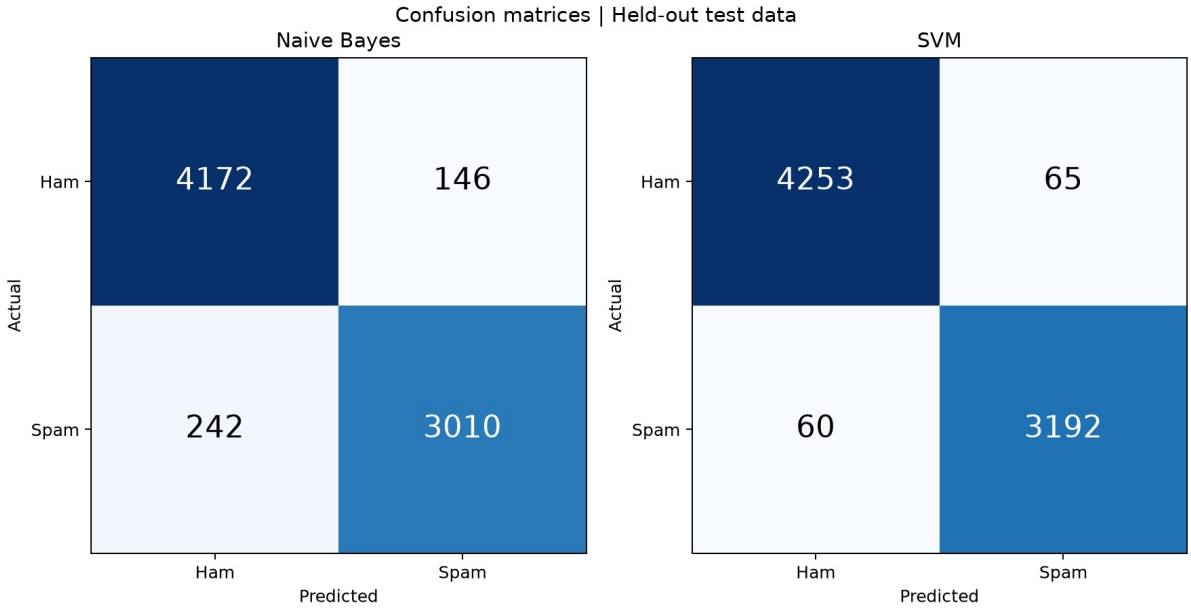


Figure 2 Confusion matrices for Naive Bayes and SVM

Table 6 Prediction error counts

Outcome	Naive Bayes	SVM
True negatives	4,172	4,253
False positives	146	65
False negatives	242	60
True positives	3,010	3,192
Total errors	388	125

SVM produced 81 fewer false positives and 182 fewer false negatives than Naive Bayes on this test partition. Its ham false positive rate is 65 divided by 4,318, or approximately 1.51 percent. Each error remains relevant when the model supports a decision about a real message.

5.3 Interpretation limits

The data mixes long emails and short SMS messages from historical corpora. The combined score can hide differences between sources. The experiment does not establish current phishing effectiveness, language specific accuracy, production calibration or resistance to novel attacks. No confidence intervals or repeated outer splits are reported.

6 Application testing

6.1 Verification after activation

The complete pytest suite was executed after selecting the newly trained models. The result was 143 passed tests with no failures. A separate check confirmed that both models load through the application prediction loader. This verifies the tested software paths, rather than guaranteeing correctness for every future input.

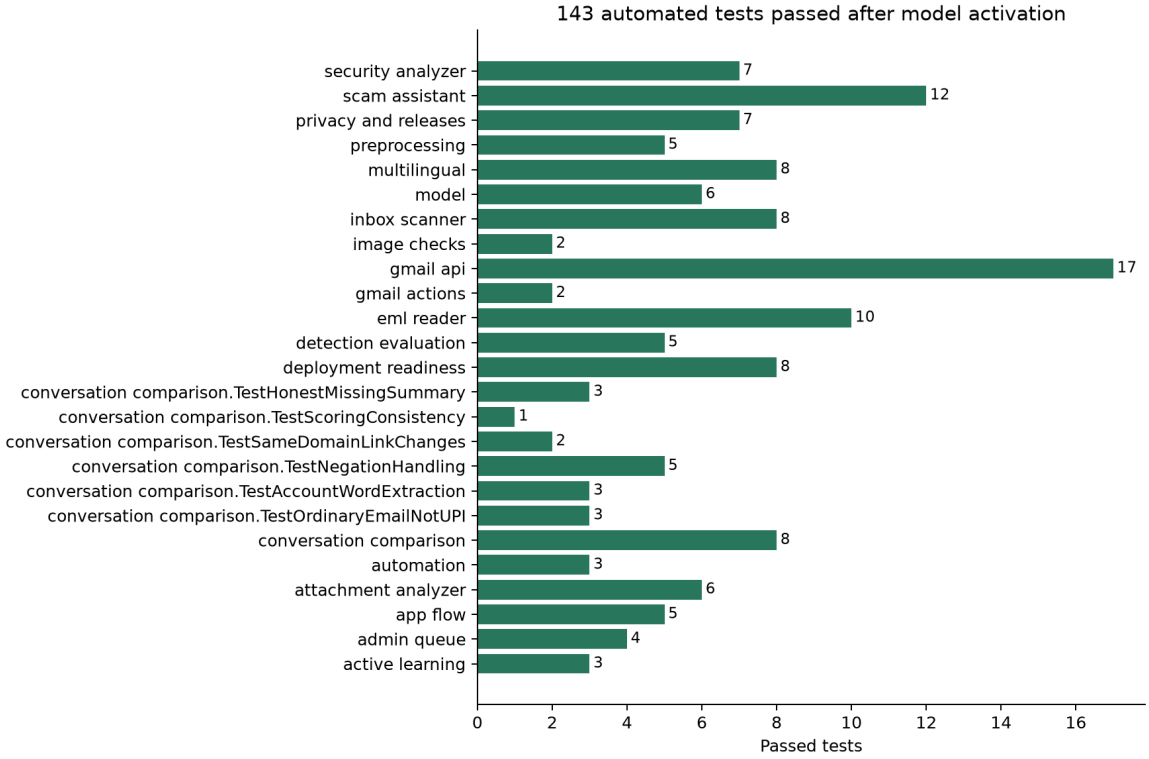


Figure 3 Automated application test results after new model activation

The graph counts passing tests by module from the post activation JUnit results. Test counts measure executed checks; they are not classification accuracy, coverage percentages or numbers of real email accounts tested.

6 Application testing continued

6.2 Coverage and boundaries

Table 7 Test coverage and boundaries

Area	What was checked
Model prediction	Model loading, known spam and ham examples, and empty input
Interface flow	Headless Streamlit interactions across the main application journey
Evidence logic	Security indicators, conversation comparison and message assessment
Parsing and images	Email parsing, attachment metadata and image handling paths
Privacy and releases	Storage isolation, feedback review, release and rollback logic
Inbox and Gmail logic	Automated checks with simulated account and service responses

The pytest fixtures use temporary private storage so tests do not read or modify saved personal account records. The JUnit record used for Figure 3 is output/training_ceas_sms_20261029_tests.xml. All 143 tests passed in the recorded post activation run.

6.3 Checks that remain outside this run

- Live Gmail authorization, real inbox access and delivery actions were not tested.
- Browser layout, responsive appearance and custom interface styling were not visually tested by the headless application tests.
- Concurrent users, sustained load, network latency and production deployment were not validated.
- The earlier 29 message synthetic scam benchmark and its timing graph were produced before this model release. They are not presented as results for the newly trained models.

6.4 Distinct forms of evidence

Application tests answer whether specified software behaviors work for their defined cases. The held out dataset measures spam classification on unseen normalized text from the supplied sources. A separate, independently labelled set of modern scams and genuine messages is still needed to assess the final hybrid warning behavior.

7 Conclusion and future scope

7.1 Conclusion

The project now has a documented two class training experiment using both supplied archives. After cleaning and normalized text deduplication, 30,277 messages were used for training and 7,570 for testing. SVM achieved 98.35 percent test accuracy compared with 94.87 percent for Naive Bayes. The application retained the new version after all 143 automated tests passed, while preserving rollback to the previous release.

The report provides source references, dataset counts, a fixed partitioning method, model settings and images generated from measured results. These records support reproduction of the historical spam experiment. They do not establish that a sender is authentic or that all present day scams can be detected.

7.2 Remaining limitations

- Historical English data provides limited evidence for current scam language, Hindi and Hinglish.
- Exact normalized deduplication does not remove every near duplicate or shared campaign.
- Attachment checks inspect metadata rather than scanning file contents for malware.
- OCR and QR extraction depend on local capabilities and do not verify linked destinations.
- Header evidence is not equivalent to independently validating sender authentication.
- The existing feedback retraining path uses its original baseline datasets; it does not automatically reuse the uploaded release training partition.

7.3 Recommended future work

Build a separately reviewed modern scam test set with genuine urgent messages, evaluate each dataset source separately, and add campaign or time separated tests. Extend representative Hindi and Hinglish coverage and assess calibration on deployment data. Any future training workflow should preserve the protected test partition and verify that application behavior remains compatible before activation.

References

Dataset sources used for this training run are cited below. The CEAS archive is one component downloaded from the Kaggle collection; the complete collection was not used. Dataset record counts in this report were measured from the supplied archives. Source pages were accessed on 29 October 2026.

[1] Alam, N. A., and collaborators. Phishing Email Dataset. Kaggle. Supplied component CEAS_08.csv, distributed in CEAS_08.csv.zip.

https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset?resource=download

[2] Almeida, T., and Hidalgo, J. (2011). SMS Spam Collection [Dataset]. UCI Machine Learning Repository. DOI 10.24432/C5CC84. Supplied file SMSSpamCollection, distributed in sms+spam+collection.zip.

https://archive.ics.uci.edu/dataset/228/sms+spam+collection

[3] scikit learn developers. TfidfVectorizer API reference.

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[4] scikit learn developers. MultinomialNB API reference.

https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html

[5] scikit learn developers. CalibratedClassifierCV API reference.

https://scikit-learn.org/stable/modules/generated/sklearn.calibration.CalibratedClassifierCV.html

[6] Streamlit documentation. App testing.

https://docs.streamlit.io/develop/concepts/app-testing

Local experiment evidence: the saved release metrics, data audit, exact training and test partitions, model manifest and post activation test record. Their filenames and integrity hashes are listed in Appendix A. Graphs in this report were generated from these local results.

Appendix A Reproducibility record

A.1 Release and experiment files

Active model version: ceas-sms-20261029. The release directory is models/versions/ceas-sms-20261029 within the project. It contains training_data.csv, test_data.csv, data_audit.json, metrics.json, release.json, spam_classifier_nb.pkl, spam_classifier_svm.pkl and tfidf_vectorizer.pkl.

The import and training script is scripts/train_uploaded_datasets.py. It accepts the CEAS archive path, SMS archive path and a new version name. The script creates a candidate; activation and regression testing are separate steps. The original input ZIP files remain unchanged.

A.2 Input integrity

CEAS_08.csv.zip

```
a6fbae6788561035e7ee0345b9767d7f
c7eddba8d7de95c7dd49961b8967de2a
```

sms+spam+collection.zip

```
1587ea43e58e82b14ff1f5425c88e17f
8496bfcdb67a583dbff9eefaf9963ce3
```

The two lines under each archive name form one continuous SHA 256 value. These hashes identify the exact input archives used for the reported run.

A.3 Test and figure provenance

The automated test record is output/training_ceas_sms_20261029_tests.xml. Figure 1 uses the recorded accuracy, spam precision, spam recall and spam F1 metrics. Figure 2 uses each classifier confusion matrix. Figure 3 is generated from the passing test cases in that post activation JUnit record.

The earlier synthetic benchmark is separate from this classifier experiment and was not used to fit these models. No private inbox messages were included in the uploaded dataset training run. Baseline files were preserved, and models/previous.json records the previous version for rollback.